DAT321/DIT843: Quality Assurance and Testing

Exam date: 2022-08-18 (pm)
Examiner: Francisco Gomes, tel. 073 856 4439, francisco.gomes@cse.gu.se
Francisco will visit the exam halls around 15:00 and 16:30.

Allowed Aid:

- English dictionary
- **NOT ALLOWED:** Anything else not explicitly mentioned above (including additional books, other notes, previous exams, or any form of electronic device: dictionaries, agendas, computers, mobile phones, etc.)

The questions in this exam refer to the **ISO 25010:2011** that categorises internal and external software quality attributes into eight characteristics, as well as the quality in use characteristics.

Important advice:

- Each question has points assigned shown in the square brackets. When the question is broken down into smaller sub-questions the part of the points for those specific sub-questions are also shown. Example: 1. [10 pts]. 1a. [2 pts] 1b. [8 pts].
- You must write clear, readable, understandable, and unambiguous answers. If we cannot easily read what you are trying to write, then we cannot positively judge your knowledge and you will most likely fail.
- Motivate your answers (a simple statement of facts not answering the question is invalid). Whenever possible refer to modules, features and information about the software product or software development process related to the questions.
- Start each question on a new paper. Sort your answers in order (by question and sub-task) before handing them in. Write only on one side of the paper.
- Write your student code on each page and put the number of the question on every paper.

The points and the corresponding grade are presented below (100 points in total):

- 00 – 49: U (Fail)
- 50 - 69: 3 (Pass)
- 70 - 89: 4 (Pass with credit)
- 90 - 100: 5 (Pass with distinction)

The exam review is scheduled for:
Date: 2022-09-06 between 14:00 – 15:00.
Location: Jupiter room 400 (Jupiter building 4th floor – Software Engineering)

1. **[Total: 35 pts]** Using the description of a software product below, answer the questions:

You work for the GPSCo company that offers an online platform for a global positioning system (GPS), similar to Google Maps. Users can type an address and obtain different options of routes to arrive in that location. Currently, the application only offers two types of routes: on foot and by car. However, the development team works with a cycle of both weekly and monthly releases for the application (see details below). We detail some of GPSCo features and processes below:

- **G1:** Users must register to use the service by creating an account with a personal e-mail and a password. Once registered, user must login to use the service.
- **G2:** The GPS application stores all of the user's search and positioning history. This allows the user to verify their past travels as well as the places visited more often. Each user only has access to their own personal positioning history.
- **G3:** GPSCo has an agreement with many other organisations to allow integration of their GPS service to third parties apps. For instance, the GPS services are heavily used by food delivery (e.g., Foodora, Uber Eats) and transport applications (e.g., Uber, or Lyft).
- **G4:** The weekly updates focus on bug fixing and improving performance of the application.
- **G5:** The monthly updates focus on delivering new features, such as new route options or new partnerships with other existing applications (see G3).
- **G6:** An important release planned is to use the GPS in embedded platforms and a variety of devices, such as the dashboard of a car, smartwatches, scooters, etc.
- **G7:** As part of an agreement with the city public transportation agency, the GPSCo application will be used to provide real-time information of the position and estimated time to arrival of busses and trams in the city centrum.
- **G8:** The app user can use their tablet, smartphone, or laptop to verify their current geographical location in a map provided by the app with a graphical interface responsive to the different screen sizes.
- **G9:** User authentication can also be done via Open Authentication (OAuth), where the user does not share their password in the application but, rather, authorises access via another service where he or she is already registered (e.g., login via their Google or Facebook account, instead of creating one with GPSCo.

Note: For the questions below, you are allowed to add features or decisions about the software development process (team size, technologies, etc.). However, any assumption related to your answer: (i) must be clearly state, and (ii) cannot contradict any of the points above.

a. [10 pts] Choose **three** software product quality characteristics, **<u>and, for each,</u>** provide examples of why they are relevant to the GPS service described above. When justifying your choices, you **must** refer to one or more of the points listed above.

There are many possible answers, so I listed a few below. The description may vary, but the question must include, per characteristic: 1-2pt: A correct quality characteristic; 2-3 an example or reasoning that is cohesive to the listed features and the characteristic itself; 1pt: a correct connection to the points listed above (G1 – G9).

Maintainability: For this you can include that the high frequency of releases (weekly) relies on a software where modifications can be made without introducing faults or issues in the product or its development process. Note that some of the listed items above explicit list the importance of adding new features or modifying the existing software product. Items: G4, G5, G6

Usability: This characteristic is relevant because the interface of the app is offered in devices with different ways of interaction or different user interface specifications. For instance, the tablet, laptop and devices have different screen resolutions, and future plans to include other devices will introduce user-specific requirements to fulfil the goals. You could also mention this under specific use cases, such as using to help with driving vehicles. Items: G8, G7

Functional Suitability: This characteristic is relevant because the positioning features are provided not only to an end-user but to other applications that rely on the correct GPS information. Items: G2, G3, G5, G7

Performance Efficiency: Relevant because the GPS service is used for real-time tracking of vehicles. Also, the user must be able to access history information which can affect future memory / storage resources for the application. Lastly, the application will be used in embedded system, which often have limited computational resources, compared to typical personal computers. Items: G2, G6, G7

Security: Relevant because the application offers user-specific information as well as it can disclose personal information about a user's location. Therefore, the correct level of access to the corresponding positioning information / history is required. Items: G1, G3, G9.

Reliability: Relevant because the GPS application is used in real-time navigation for, e.g., users as well as the public transportation. Therefore, the system must function as expected and consistently throughout the day. Items: G2, G6, G7

Portability: Relevant because the GPS application is intended to be used in different hardware devices. Items: G6, G8

Compatibility: Relevant because GPSCo has various agreement with third parties, such that the GPS application must be able to properly share the correct and suitable positioning information to its user across the different applications connected to the service. Items: G3, G7, G9.

b. [12 pts] Choose two external quality and two internal quality metrics to assess the software product described above. For each chosen metric you must: i) describe and justify which quality characteristic/attribute is associated with your chosen metric, ii) choose one of the listed features/characteristics above that the metric can helps with.

There are many acceptable metrics. 3-4 pts per metric. The key aspect is that: [2-3pt] (i) the description and justification are correct, [1-2pts] (ii) the metric must be connected to one of the items listed above (G1-G9):

External quality measures:

Probability of Failure on Demand (POFOD). (i) The purpose is to measure the proportion of requests that fail based on the total amount of requests made; both failures and crashes are counted as failed requests. Therefore, refers to Reliability. (ii) can be used with items G2, G3, G7.

Rate of Occurrence of Failures (ROCOF). (i) ROCOF conveys the number of failed requests in relation to a time interval in which those requests were sent (hours, days, weeks, etc.). Conveys Reliability. (ii) can be used with items G2, G3, G7.

Availability. (i) Availability conveys the probability to access and complete tasks using the application / service. Availability is measured based on the mean uptime and downtime of the application. Refers to Reliability. (ii) can be used with items G2, G3, G7.

Number of failures (unexpected outcomes). (i) Can be measure on components connected to the expected behavior of the software such as Booking Handler or Clinic Handler. (ii) Measures the number of test cases where the actual output did not match an expected output when executing the system. Should indicate that a fault exists. Failures are connected to reliability or functional suitability. (iii) can be used in nearly all items.

Latency and jittering. (i) Measures the time to process a request in the system (i.e., send the request and receive a response), whereas jittering is the variation around that time after performing multiple requests. Connected to Performance Efficiency. (ii) Can be used by items G1, G2, G3, G7, G8, G9.

Other examples of measures are: Mean time between failures, throughout (if mentioned in terms of tasks, and not in terms of requests, otherwise that is latency), uptime, downtime, etc.

Internal Quality measures:

Lines of code: (i) Measure the size of the software in terms of the amount of code written. Refers to maintainability. (ii) Can be used mainly in items G5 and G6.

Cyclomatic Complexity: (i) Measures the number of independent paths that can be executed when running the software. Refers to maintainability, functional suitability and performance efficiency. (ii) Can be used mainly in items G2, G5 and G6.

Cognitive Complexity: (i) Measures the level and depth of nested code that can hinder data and flow dependencies. Refers to maintainability, functional suitability and performance efficiency. (ii) Can be used mainly in items G2, G5 and G6.

Coupling: (i) Measures the number of dependencies between different software artefacts (e.g., code, packages, components, methods, classes etc.). Refers to maintainability. (ii) Can be used mainly in items G5 and G6. Could also be argued for G3, G7.

Other examples of internal measures are: Halstead complexity, code churn, number of commits, number of pull requests, block depth, or other measures obtained from the static artefacts of the software product.

c. [8 pts] GPSCo wants to promote sustainability in the product that we are creating. Using the Sustainability Awareness Framework, you must (i) **choose or suggest two functionalities for the GPS service** and explain the (ii) immediate, (iii) enabling and (iv) structural effects of those features. You must also indicate the **corresponding type/dimension of sustainability** in which those effects occur.

There are many possible correct answers. Each suggestion receives 4 pt, based on: [2-3 pts] Having a correct description and a cohesive connection between the immediate, enabling and structural effects; and [1-2pts] for indicating the correct dimension (environmental, technical, economic, social or individual) for each of those effects. Some examples:

Suggested feature: Design and include colourblind friendly graphical user interface (GUI). immediate: improve usability for end-users that are colourblind (individual); enabling: attract and allow more users with disabilities to use the app (social); structural: promote inclusion and diversity and inspire other apps to also include colourblind friendly GUI (social);

Suggested feature: Include routes for users riding bicycles and include suggestions based on weather predictions. Immediate: Encourage users to ride bikes for trips that could not be done by foot and can be avoided by car (environment); enabling: End-users get more exercises by choosing between walking and cycling (individual); structural: Reduce the changes of traffic jams by having fewer users using cars (environment).

Suggested feature: Include suggestions of well-reviewed restaurants or bars in the area around the user's destination, home or work. Immediate: Create a partnership with existing platforms that offer review and information about local establishments (economical) Enabling: Offer end-users options for socialising or entertainment in their areas (individual), Structural: Stimulate the economy in different areas throughout the city (economical)

d. [5 pts] The GPSCo software product can run on three operating systems (MacOS, Windows and Linux), and can be accessed via three different web browsers (Firefox, Google Chrome and Microsoft Edge) designed for two screen resolutions (1920x1200 and 1280x800). Your manager Sara states that there are not enough time or resources to run all possible test cases. Propose a technique that can be used to minimise the testing costs. Justify your choice and explain the trade-off of using your proposed technique.

Since testing all combinations of interacting features is not feasible, we can propose to Sara to use Combinatorial Interaction Testing (CIT), where we guarantee the exercise of all t-wise interaction between each varying factor (1-2 pts). Typically, CIT focuses on all pairwise interactions to reduce the number of combinations verified, but systematically guaranteeing that a subset of those is always verified (2-3 pts).

2. **[Total: 45 pts]** Considering the code below and the techniques to design and evaluate test cases answer the questions below:

```
// Identifies and returns the two highest values in an array of integers.
// If the array has less than two elements, the method throws an
//   exception. Since this is a Java implementation, you can assume that
//   the array does not have any null value.
//
// Input : An array of integers
// Output: An array with two integers, where the first element is the
//         highest integer value in the input array, and the second
//         element is the second highest integer.
1. public int[] getTwoHighest(int[] numbers) throws Exception {
2.     if(numbers.length < 2){
3.        throw new Exception("Array must have at least two elements.");
4.     }
5.     int topValue   = numbers[0];
6.     int secondBest = numbers[1];
7.     for (int i = 1; i < numbers.length; i++) {
8.         int currentNumber = numbers[i];
9.         if(currentNumber > topValue) {
10.             secondBest = topValue;
11.             topValue = currentNumber;
12.         } else if(currentNumber > secondBest){
13.             secondBest = currentNumber;
14.         }
15.     }

16.     int[] topTwo = new int[2];
17.     topTwo[0] = topValue;   // first element is the highest value
18.     topTwo[1] = secondBest; // second highest in second array index.
19.     return topTwo;
20. }
```

a. [15 pts] Write **at least four equivalence partition** classes that can be used to test the method above. Use the method under test (specification or code) to justify your choice of partition classes.

Each correct partition class receives [3-4 pts]. It is important to define the partitions in terms of the input parameters of the function in relation to the output (not the other way around).

Here, you must define them in terms of the composition of the elements in the array. A few examples of partitions are listed below (the partition IDs below are important to be referenced in the future questions):

P1: Arrays with less than two elements.

Output: Triggers an exception. (Careful not to include redundant partitions to this one, e.g., arrays with one element or empty arrays – they cause the same outcome).
Reason: Verifies the first if statement that throws exception.

P2: Arrays where the highest element comes in an index before the second highest.
Output: Returns a valid output array.
Reason: Verifies the correctness of the base case (and similar ones) where the topValue is found before the second highest value (lines 5—9).

P3: Arrays where the highest element comes in and index after the second highest.
Output: Returns a valid output array.
Reason: Verifies the correctness of the conditional that swaps between a top value and the second highest (lines 9 and 12).

P4: Arrays where the top two values are equal.
Output: Returns a valid output array.
Reason: Verifies the correctness of the relational operation in the conditionals on lines 9 and 12.

b. [15 pts] Using Boundary Value Analysis (BVA), write **test cases that reveal at least <u>five boundaries</u>** (input and expected output) of the partition classes that you identified above. Indicate the corresponding boundary listed.

Each boundary must include two test cases (one with input from each partition) that are "close" to each other, e.g., by adding or removing one element in the array. Each correct pair of tests yields 3 points. For a correct test you must include: (i) an actual input value, (ii) an expected output, (iii) and the corresponding partitions. Some examples:

Boundary 1: between P1 and P2
P1: TC1: numbers = {0}; output = Exception: "Array must have at least two elements."
P2: TC2: numbers = {0, 1}; output = {0, 1}

Boundary 2: between P1 and P3
P1: TC3: numbers = {}; output = Exception: "Array must have at least two elements."
P3: TC4: numbers = {88, 100}; output = {100, 88}

Boundary 3: between P1 and P4
P1: TC5: numbers = {}; output = Exception: "Array must have at least two elements."
P4: TC6: numbers = {-50, -50}; output = {-50, -50}

Boundary 4: between P2 and P3
P2: TC7: numbers = {100, 20, 50, -10}; output = {100, 50}
P3: TC8: numbers = {4501, 1209, 7820, -10000}; output = {7820, 4501}

Boundary 5: between P2 and P4
P2: TC9: numbers = {0, -5, -10, -500}; output = {0, -5}
P4: TC10: numbers = {0, 0, 0, 0}; output = {0, 0}

c. [7 pts] Regarding mutation testing, describe at least two different mutation operators that can be applied to the method above. Choose one or more lines of code above to illustrate and apply each mutation operator by indicating i) the corresponding line of code and ii) the resulting mutant in that line.

There are different operators that can be used, but some of them cannot. For instance, there are no conditionals to be replaces (||, &&), and the only arithmetic expression in the code is the increment in the loop --- which is typically another mutation operator such as changing an increment to a decrement. So, for this example, we can use the Relational Operation Replacement (ROR) and the Assignment Operation Replacement (AOR). [3-4 points per correct description of the operator and mutation of the code]

The ROR changes a relational operator such as >, <, !=, >=, to another one. That is applicable to many lines, such as i) line 2; ii) `if(numbers.length <= 2)`

The AOR changes an assignment operator such as =, +=, \=, etc to another one. That is applicable to many lines, such as i) line 10 or 11; ii) `secondBest += topValue`
Note that if you choose lines where the variable is being declared (e.g., lines 8), then the AOR will create a compilation error, and that is not a mutant.

d. [8 pts] For each mutant created above, explain why your mutant is a good choice for mutation testing.

The answer depends on your choice above. Make sure to create a cohesive argument between the code mutation and the algorithm shown in the code. [4pts per mutation operator]. It optional but very beneficial if you have examples of test cases that can kill (or not) the mutant.

The mutation using ROR in line 2 is a good choice because it allows us to see if there is a test case verifying the boundary of the partition that triggers the exception, since an array with two elements should not trigger an exception.

The mutation using AOR in line 10 is a good choice because it allows us to test whether the correct value of the variable secondBest is being stored, rather than being lots or accumulated with zero (e.g., this mutant would not be killed with test case TC10 above).

3. **[Total: 20 pts]** You are asked to test a method that receives as input two sets of strings `setA` and `setB` and returns the union between those two sets (`setA` ∪ `setB`). Answer the questions below:

a. [8 pts] Using Property-based testing (PBT) write down **at least three properties** for verifying the union method described above. You should write the properties using natural language.

[2-3 pts per property]. The property must be defined in terms of the input parameter, since this will be the one that the PBT tool will automatically generate. Make sure to not be redundant, but many of the guidelines seen to define good properties are also applicable in set theory (commutative, idempotent). Some examples:

Property 1: The union of a set with itself results in the input set:

- `union(setA, setA) == setA`

Property 2: The union between setA and setB is the same as the union between setB and setA

- `union(setA, setB) == union(setB, setA)`

Property 3: The union between setA and an empty set results in setA. (different than property 1, since setA is not necessarily empty)

- `If setC = {}; union(setA, setC) == setA`

Property 4: The union can be applied many times to the same pair of sets and it will not change the result of the first union done.

- `union(setA, setB) == union(union(setA, setB), union(setA, setB))`

b. [7 pts] Explain at least two differences between Property-based Testing and Search-based Software Testing techniques.

[3-4 pts per difference] Examples of difference:

- Search-based testing relies on a fitness function and a metaheuristic to guide the generation of the test inputs, whereas the PBT tools generate random values that fulfil constraints defined in the properties themselves (e.g., empty set, or set with specific sizes).
- SBST can be used to generate a set of test cases, depending on how you represent / define your solution, whereas PBT focuses mainly on generating test input / test cases.
- Fitness functions can be designed to use any information (or combination of them) from the test artefact (input, output, length of the test), whereas PBT is defined in terms of constraints between the acceptable input values.

c. [5 pts] Your colleague Alex states that both Boundary Value Analysis and Property-based Testing are Black-box testing techniques. Do you agree or disagree with Alex? Justify your answer.

Agree with Alex. Unlike white-box techniques, both BVA and PBT techniques do not require code or structural information from the program under tests. Code information can be useful for PBT and BVA, but it is not required. In fact, they only require the definition of the input and expected output to be used by testers.